Industrial AI Process Logic

- Repository: https://github.com/j-avdeev/industrial-ai-process-logic.
- Track 1 pipeline for semiconductor process next-step, completion, and anomaly tasks.
- Built as a reproducible baseline, training, Leonardo, and packaging workflow.

Problem

- Process routes are long, family-specific, and constrained by hard fabrication rules.
- A useful system must predict plausible next steps, complete partial routes, and flag invalid sequences.
- The jury needs runnable code, honest metrics, and evidence that the final artifacts match the run.

Core Approach

- Treat full process steps as tokens and condition every prediction on MOSFET, IGBT, or IC family.
- Use official grammar and validator logic as a reliability layer for anomalies and candidate filtering.
- Combine retrieval, n-gram beam search, validator penalties, and optional transformer checkpoint reranking.

What Runs Locally

- ‘industrial_ai.smoke_pipeline‘ creates dev inputs, predictions, metrics, package manifests, and audits.
- ‘industrial_ai.infer‘ writes ‘nextstep.csv‘, ‘completion.csv‘, and ‘anomaly.csv‘.
- Local smoke checks are intentionally small; they prove the pipeline, not final leaderboard quality.

Leonardo Path

- Generate 50k-150k valid sequences per product family with the official generator.
- Train tiny, small, and medium transformer checkpoints on CUDA.
- Select a checkpoint reranker, run official inference, and package submissions plus evidence.

Verification

- Preflight validates raw data, eval staging, PyTorch, CUDA, and expected input schemas.
- Corpus, checkpoint, validation, final-audit, and returned-package checks bind hashes and thresholds.
- Source-bundle proof prevents script or Python source drift between upload, training, and packaging.

Honest Status

- Mature: repo, MIT license, README, REPORT, dependency manifest, Slurm scripts, packaging, and verifiers.
- Pending external result: official eval files, Leonardo training run, returned package, and final scores.
- Tiny smoke completion exact match can be 0.0000; final quality depends on the large Leonardo corpus.

Next Step

- Stage official eval CSVs and run the Leonardo readiness command.
- Submit the 50k or 150k-per-family full pipeline with CUDA training.
- Verify ‘artifacts/leonardo_return_packet.zip‘ before uploading the final CSV package.